



Vetorização no NumPy e Pandas: axis, Broadcasting, apply e map sem Laços

Estruturas Vetorizadas

NumPy representa dados numéricos como **arrays vetorizados**, permitindo aplicar funções matemáticas em blocos inteiros de dados. Pandas usa essas estruturas internamente para gerenciar tabelas de forma eficiente, eliminando a necessidade de loops explícitos.

```
import numpy as np  
x = np.array([1, 2, 3, 4])  
np.mean(x)
```

Saída: 2.5

Criando uma Tabela com Pandas

Vamos criar uma tabela com peso, altura e identificação de pessoas. Cada coluna é uma série de valores, e o DataFrame organiza tudo em formato tabular.

```
import numpy as np
import pandas as pd
data = {
    "weight": [60, 72, 57, 90, 95, 72],
    "height": [1.75, 1.80, 1.65, 1.90, 1.74, 1.91],
    "subject": ["A", "B", "C", "D", "E", "F"]
}
df = pd.DataFrame(data)
```

Resultado

Tabela com 6 linhas e 3 colunas: weight, height e subject

Selecionando Colunas Numéricas

Antes de calcular estatísticas, é essencial selecionar apenas as colunas numéricas. Isso evita erros ao processar colunas de texto.

```
num_cols = df.select_dtypes(  
    include="number"  
)
```

Por que filtrar?

- Evita erros de tipo
- Isola dados quantitativos
- Prepara para cálculos
- Melhora performance

Médias por Coluna (Vetorizado)

Cálculo Automático

O pandas aplica `mean()` sem necessidade de loops explícitos

Resultado Imediato

Retorna uma série com um valor por coluna instantaneamente

Processamento Vetorizado

Operação em nível de C, garantindo máxima eficiência

```
num_cols.mean()
```

Saída:

weight 74.333333

height 1.791667

Operações com Axis em NumPy

Use o parâmetro `axis` para controlar a direção do cálculo em matrizes multidimensionais.

axis=0

Processa **por coluna**, verticalmente

axis=1

Processa **por linha**, horizontalmente

```
m = num_cols.to_numpy()
```

Mínimo por Linha e por Coluna

Por Linha (axis=1)

Encontra o menor valor em cada linha, comparando as colunas entre si.

```
np.min(m, axis=1)
```

Saída:

```
array([1.75, 1.8, 1.65, 1.9, 1.74, 1.91])
```

Por Coluna (axis=0)

Encontra o menor valor em cada coluna, comparando todas as linhas.

```
np.min(m, axis=0)
```

Saída:

```
array([57., 1.65])
```

Operações Elemento a Elemento

Alinhamento Automático

NumPy e pandas alinham vetores compatíveis automaticamente

Operações Elemento a Elemento

Cálculos aplicados em cada posição correspondente

Broadcasting

Cria novas variáveis sem loops explícitos

```
df["height"] ** 2
```

Saída:

```
0    3.0625
1    3.2400
2    2.7225
3    3.6100
4    3.0276
5    3.6481
```

Name: height, dtype: float64

O expoente 2 (escalar) é aplicado a cada elemento da coluna. Entre duas colunas, o pandas alinha os valores pelo índice.

Cálculo Derivado sem Loop

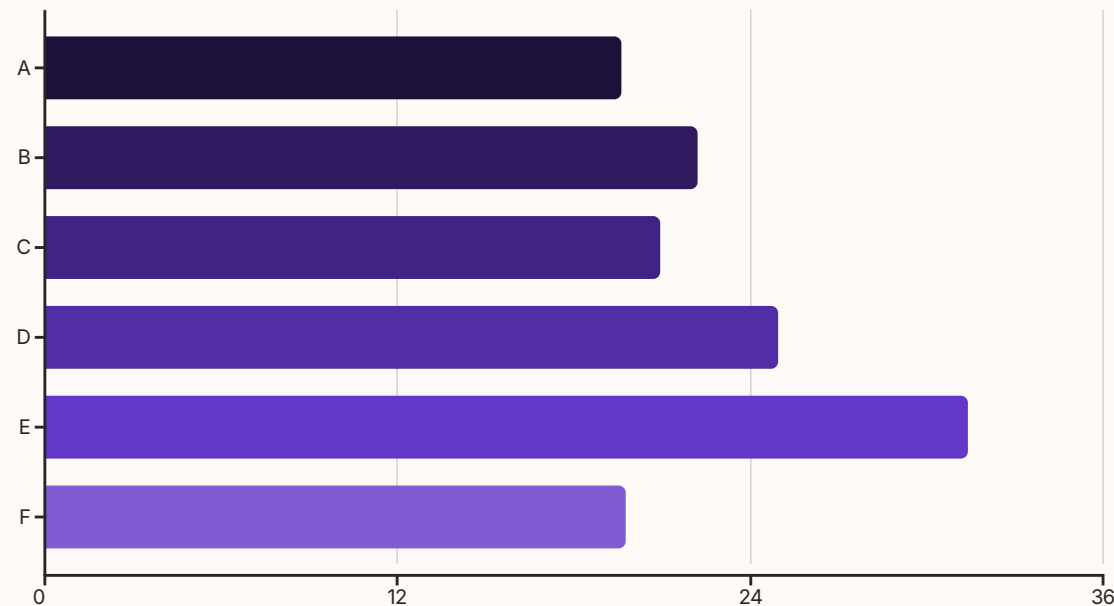
Podemos combinar colunas para criar novas métricas de forma vetorizada. Aqui calculamos o **Índice de Massa Corporal (IMC)**, onde cada linha é processada automaticamente sem iteração explícita.

Fórmula do IMC

$\text{IMC} = \text{peso} / (\text{altura}^2)$

Implementação

```
df["bmi"] = df["weight"] /  
(df["height"] ** 2)
```



Aplicar Função a Várias Colunas

Quando precisamos aplicar uma função a colunas, linhas ou elementos, o pandas oferece apply e map, sem laços explícitos.

Por coluna: apply

```
cols = ["weight", "height"]
df[cols].apply(np.mean)
# weight    74.333333
# height    1.791667
```

Por linha: apply com axis=1

```
df.apply(lambda r: r["weight"] /
          r["height"]**2, axis=1)
# mesmo resultado do IMC,
# porém mais lento
```

Por elemento: map

```
df["subject"].map(str.lower)
# 0    a
# 1    b ...
```

❏ **Dica:** Use apply/map quando não existir uma operação vetorizada pronta. Quando ela existir (como no IMC), a vetorização é sempre mais rápida.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Introdução essencial à programação e ao pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Guia completo das principais ferramentas para análise de dados em Python, incluindo NumPy, Pandas e bibliotecas de visualização.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda fundamentos de ciência de dados construindo algoritmos e ferramentas do zero com Python.